```
import os
from google import genai
from dotenv import load_dotenv

def main():
    load_dotenv()
    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        print("Error: GEMINI_API_KEY not found.")
        return

    client = genai.Client(api_key=api_key)

    model_id = "gemini-2.5-flash"

    # Define your persona
    persona = "You are straight forward no suger coated
chatbot that answers questions in a concise manner. You
do not provide any additional information or context
beyond what is asked. You do not engage in small talk or
provide opinions. Your responses are direct and to the
point."

    print(f"--- Gemini Chatbot Started ---")

    # FIX: Pass system_instruction directly as an argument to
the chat creator
    # Some SDK versions prefer it here to avoid the JSON
'field not found' error
    chat = client.chats.create(
        model=model_id,
        config={
            'system_instruction': persona,
            'temperature': 0.7
        }
    )

    while True:
        try:
```

```
            user_input = input("You: ").strip()
            if user_input.lower() in ["exit", "quit"]:
                break

            if not user_input:
                continue

            response = chat.send_message(user_input)
            print(f"Bot: {response.text}\n")

        except Exception as e:
            # If the error persists, it's often a naming
convention issue (system_instruction vs systemInstruction)
            print(f"An error occurred: {e}")
            break

if __name__ == "__main__":
    main()
```

whats the next project

Nice, this is a solid base—you've already crossed the "talking to an LLM properly" stage. Now don't stay here too long; a plain chatbot won't teach you much more.

### 👉 Your next project should be:

## RAG PDF Chatbot (Retrieval-Augmented Generation) ⭐

This is where things actually start getting real.

---

## 💡 What you'll build

A chatbot that can:

- Take a PDF (notes, docs, etc.)
- Understand its content
- Answer questions **based only on that PDF**

---

## 🧠 Why this is the right next step

Right now your bot:

- ❌ Hallucinates

- ❌ Has no memory of external knowledge

RAG fixes that:

- ✅ Grounded answers
- ✅ Real-world use case (used in companies)
- ✅ Core GenAI skill

---

## 🪜 Project Breakdown (keep it simple)

### 1. Load PDF

Use:

- `PyPDF` or `pdfplumber`

---

### 2. Chunk the text

Split into small pieces:

```
chunk_size = 500
overlap = 50
```

---

### 3. Convert to embeddings

Use:

- Gemini embeddings OR
- FAISS-compatible embeddings

---

### 4. Store in vector DB

Use:

- FAISS (best for beginners)

---

### 5. Retrieval step

When user asks:

- Convert query → embedding

- Find top 3–5 similar chunks

---

### 6. Final prompt to LLM

Instead of:

```
User: What is X?
```

You do:

```
Context: <retrieved chunks>
Question: What is X?
Answer only from context.
```

---

## ⚙️ Tech Stack (don't overcomplicate)

- Python
- FAISS
- Gemini API (you're already using it)

---

## 🚀 Stretch Goals (after basic version works)

- Add **chat memory**
- Add **multiple PDFs**
- Add **source citations**
- Build simple UI (Streamlit)

---

## ⚠️ Important advice (don't skip this)

You asked earlier about LangChain/LangGraph.

👉 **Do NOT use them yet.**
Build this **from scratch first**:

- You'll actually understand embeddings + retrieval
- Otherwise you'll just "call functions blindly"

---

## 🧭 After this project

Then you move to:

1. **Semantic Search Engine**
2. **Multi-doc RAG**
3. **Agent (tool-using bot)**

---

If you want, I can guide you like:

- "Day 1: PDF → text"
- "Day 2: chunking"
- "Day 3: embeddings + FAISS"

Say the word and we'll build it step by step.

start

Perfect. Let's build this like an engineer, not copy-paste chaos.

---

# 🚀 Day 1: PDF → Clean Text

## 🎯 Goal

By the end of today:

- Load a PDF
- Extract all text
- Print it cleanly

No embeddings. No AI yet. Just **data pipeline**.

---

## 🧱 Step 1: Install dependencies

Run:

Bash

## 📁 Step 2: Project structure

Keep it clean:

```
rag-pdf-chatbot/
├── main.py
├── sample.pdf
├── .env
```

---

## 🧠 Step 3: Write the code

Here's your **Day 1 code**:

```python
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""

    for page_num, page in enumerate(reader.pages):
        page_text = page.extract_text()

        if page_text:  # avoid None pages
            text += page_text + "\n"

    return text


def main():
    file_path = "sample.pdf"  # put your PDF here

    text = extract_text_from_pdf(file_path)

    print("---- Extracted Text Preview ----\n")
    print(text[:1000])  # preview first 1000 chars


if __name__ == "__main__":
    main()
```

---

## ✅ What you should check

Run it:

Bash

You should see:

- Text printed
- No weird crashes
- Some formatting issues (normal)

---

## ⚠️ Common issues (don't get stuck here)

- Empty output → PDF might be scanned (image-based)
- Weird spacing → normal for PDFs
- Crash → check file path

---

## 🔍 Quick sanity test

Ask yourself:

- Can I see actual readable text? ✅
- Does it roughly match PDF content? ✅

If yes → you're done for Day 1.

---

## 🧠 What you just learned (important)

- PDFs are messy
- Text extraction is NOT perfect
- This is why chunking later matters

---

## 👉 Your task

1. Run this code
2. Use **your own PDF** (notes, book, anything)
3. Tell me:
   - "works" ✅
   - OR paste error

---

## ⏭️ Tomorrow (Day 2)

We'll do:

👉 **Chunking (the most underrated part of RAG)**

That's where most beginners mess up.

---

Reply with:

- "done"
- or error screenshot/code

I'll push you to the next step.

why we need to chunck the pdf

Short answer: **because LLMs and embeddings can't handle huge text effectively.**

---

## 🧠 The real reasons (no fluff)

### 1. ❌ Token limits (hard constraint)

Models like Gemini don't accept infinite text.

If your PDF is:

- 50 pages → too big
- 100 pages → impossible

So you **must split it**.

---

### 2. ❌ Retrieval doesn't work on big blobs

If you store the whole PDF as one chunk:

User asks:

> "What is gradient descent?"

Your system:

- Compares question with entire book ❌
- Similarity becomes weak ❌
- Returns irrelevant stuff ❌

---

### 3. ✅ Chunking makes search precise

When you split into small pieces:

- Each chunk = focused idea
- Embeddings = meaningful
- Retrieval = accurate

So:

- Question → matches **specific paragraph**, not whole book

---

### 4. ❌ Cost + performance

Without chunking:

- Bigger input → slower
- More tokens → more cost

With chunking:

- Only top 3–5 chunks sent → fast + cheap

---

### 5. ✅ Better final answers

Instead of:

> dumping entire PDF into prompt

You do:

```
Context: [only relevant chunks]
Question: ...
```

→ Cleaner, more accurate answers

---

## 🔥 Simple analogy

Think of it like searching:

- ❌ Whole PDF = one giant paragraph
- ✅ Chunked = indexed Google search

You don't search the internet as one file—you search **indexed pieces**.

---

## ⚠️ Important insight (most people miss this)

Chunking is NOT just splitting randomly.

Bad chunking = bad RAG
Good chunking = powerful system

This is where beginners silently fail.

---

## 🧭 What's next

Now that you get *why*:

👉 Day 2 = **How to chunk properly (not just split blindly)**

I'll show you:

- chunk size
- overlap
- mistakes to avoid

---

Say "**next**" and we'll build chunking properly.

before these rag we need to do semantic-search-faiss

Short answer: **No—you don't *need* to do semantic search separately first.**
But… doing a **mini version of it inside your RAG build** is the smartest move.

---

## 🧠 How these actually relate

Think of it like this:

- **Semantic Search (FAISS)** = *engine*
- **RAG** = *full car (engine + steering + UI)*

So yeah, semantic search is a **part of RAG**, not a completely different path.

---

## ⚖️ Two ways you could go

### Option A (what most beginners do ❌)

- Learn FAISS separately
- Then try RAG later
  → Feels disconnected, boring, slow

---

### Option B (what you should do ✅)

- Build RAG
- Learn FAISS **inside it**

→ You understand:

- *why* retrieval matters
- not just *how* to code it

---

## 🧭 What I recommend for you

Given your current level (you just built chatbot):

**👉 Do NOT pause for a separate project**

Instead:

### Build RAG like this:

**Day 1** → PDF extraction ✅ (you did)
**Day 2** → Chunking
**Day 3** → Embeddings + FAISS (**this = semantic search**)
**Day 4** → Retrieval
**Day 5** → LLM response

---

## 🔥 Important mindset shift

Don't think:

> "Should I learn FAISS first?"

Think:

> "I'll learn FAISS exactly when I need it"

That's how real engineers work.

---

## ⚠️ When should you do a separate semantic search project?

Only if:

- You want deeper understanding later
- You want to optimize retrieval systems

👉 That's **after your first RAG works**

---

## 🧩 What's coming next

Tomorrow (your next step):

👉 **Chunking (this matters more than FAISS actually)**

Most people mess up here and blame embeddings later.

---

If you're ready, say:
👉 **"next"**

We'll build chunking properly (with code, not theory).

```
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text=""
    for page_num,page in enumerate(PdfReader.pages):
        page_text=page.extract()
        if page_text:
            text+=page_text +'\n'
```

```
    return text

def main():
    filepath="sample.pdf"
    print("your pdf :")
    text=extract_text_from_pdf(file_path=filepath)
    print("your text is: \n ")
    print(text[:1000])
main()
```

You're close—but this code won't work as written. There are a few **critical mistakes**.

Let's fix them cleanly.

---

## ❌ What's wrong

### 1. Wrong object used

Python Run

- `PdfReader.pages` ❌ (class-level, wrong)
- You already created `reader`

---

### 2. Wrong method name

Python Run

- Should be `extract_text()`

---

### 3. Minor naming inconsistency

- `file_path` vs `filepath` → works but sloppy

---

## ✅ Corrected code

```python
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""

    for page_num, page in enumerate(reader.pages):
        page_text = page.extract_text()

        if page_text:
            text += page_text + "\n"

    return text


def main():
    file_path = "sample.pdf"

    print("Your PDF text preview:\n")

    text = extract_text_from_pdf(file_path)

    print(text[:1000])


if __name__ == "__main__":
    main()
```

---

## 🔍 Why your version failed (important to understand)

- `PdfReader.pages` → doesn't exist like that
- `page.extract()` → not a real method
- So Python would throw errors like:
  - `AttributeError`
  - or just crash silently depending on environment

---

## ✅ What you should do now

1. Fix the code
2. Run it
3. Confirm:
   - You see readable text
   - No errors

## ⚠️ If output is empty

Your PDF is likely:

- scanned (image-based)

Then you'll need OCR later (ignore for now)

---

## ⏭️ Next step

Once this works:

👉 say "**done**"

We move to:

### Day 2: Chunking (this is where real learning starts)

```
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text=""
    for page_num,page in enumerate(reader.pages):
        page_text=page.extract_text()
        if page_text:
            text+=page_text +'\n'
    return text

def main():
    filepath="sample.pdf"
    print("your pdf :")
    text=extract_text_from_pdf(file_path=filepath)
    print("your text is: \n ")
    print(text[:1000])
main()
```

This version is basically **correct and will work** 👍
Just a couple of small things to clean up so you don't build bad habits.

---

## ✅ What's good

- ✔ Correct use of `PdfReader`
- ✔ Proper loop over `reader.pages`
- ✔ Using `extract_text()` (fixed earlier mistake)
- ✔ Previewing output → good debugging habit

---

## ⚠️ Minor improvements (do these)

### 1. Use proper entry point (important habit)

Right now:

Python Run

Better:

```python
if __name__ == "__main__":
    main()
```

Why:

- Prevents issues when you import this file later (you *will* in RAG)

---

### 2. Clean variable naming (consistency matters)

```python
filepath="sample.pdf"
text=extract_text_from_pdf(file_path=filepath)
```

Just use one style:

```
file_path = "sample.pdf"
```

Python Run

---

### 3. Optional: Add page numbers (useful later)

Python Run

This helps later for:

- citations
- debugging retrieval

---

## ✅ Clean final version

```python
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""

    for page_num, page in enumerate(reader.pages):
        page_text = page.extract_text()

        if page_text:
            text += f"[Page {page_num}]\n{page_text}\n"
```

Python Run

```python
def main():
    file_path = "sample.pdf"

    print("Your PDF:\n")

    text = extract_text_from_pdf(file_path)

    print("Extracted text:\n")
    print(text[:1000])


if __name__ == "__main__":
    main()
```

---

## 🎯 Where you are now

You've successfully done:

- ✅ PDF → text extraction
- ✅ Clean pipeline start

This is **exactly where most beginners quit or get confused**—you didn't.

---

## ⏭️ Next step (important one)

Now we move to:

# Day 2: Chunking (core of RAG)

This is where your system either:

- becomes powerful ✅
- or useless ❌

Say:
👉 **“next”**

We’ll build chunking properly (not just random splitting).

```
from pypdf import PdfReader

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
    text=""
    for page_num,page in enumerate(reader.pages):
        page_text=page.extract_text()
        if page_text:
            text+=page_text +'\n'
    return text

def main():
    filepath="sample.pdf"
    print("your pdf :")
    text=extract_text_from_pdf(file_path=filepath)
    print("your text is: \n ")
    print(text[:1000])

if __name__ == "__main__":
    main()
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf> python main.py
your pdf :
invalid pdf header: b'Simpl'
EOF marker not found
Traceback (most recent call last):
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf\main.py", line 18, in <module>
    main()
    ~~~~^^
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf\main.py", line 15, in main
    text=extract_text_from_pdf(file_path=filepath)
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
```

```
extract_text_from_pdf\main.py", line 4, in
extract_text_from_pdf
    reader = PdfReader(file_path)
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 149, in __init__
    self._initialize_stream(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 171, in _initialize_stream
    self.read(stream)
    ~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 661, in read
    self._find_eof_marker(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 767, in _find_eof_marker
    line = read_previous_line(stream)
  File "C:\Python313\Lib\site-packages\pypdf\_utils.py",
line 330, in read_previous_line
    raise
PdfStreamError(STREAM_TRUNCATED_PREMATURELY)
pypdf.errors.PdfStreamError: Stream has ended
unexpectedly
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf> python main.py
your pdf :
invalid pdf header: b'Simpl'
EOF marker not found
Traceback (most recent call last):
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 20, in <module>
    main()
    ~~~~^^
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 15, in main
    text=extract_text_from_pdf(file_path=filepath)
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 4, in
extract_text_from_pdf
    reader = PdfReader(file_path)
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 149, in __init__
```

self._initialize_stream(stream)
~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 171, in _initialize_stream
self.read(stream)
~~~~~~~~~~^^^^^^^^
File "C:\Python313\Lib\site-packages\pypdf_reader.py", line 661, in read
self._find_eof_marker(stream)
~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 767, in _find_eof_marker
line = read_previous_line(stream)
File "C:\Python313\Lib\site-packages\pypdf\_utils.py", line 330, in read_previous_line
raise PdfStreamError(STREAM_TRUNCATED_PREMATURELY)
pypdf.errors.PdfStreamError: Stream has ended unexpectedly
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf>

Good—this is a **real debugging moment**, not a coding mistake.

Your code is fine.
👉 The **PDF file is the problem**, not your logic.

---

## ❌ What this error actually means

```
invalid pdf header: b'Simpl'
EOF marker not found
```

This tells you:

- File is **not a valid PDF**
- Or it's **corrupted / incomplete**

That `b'Simpl'` is a big clue:
👉 Your file probably starts with text like:

```
Simple something...
```
~~~~~~~~~~~~~~~~~~~~~~~

instead of a real PDF header:

```
%PDF-1.4
```

---

## 🔍 Most common causes

### 1. ❌ You didn't download the file properly

Example:

- You saved a webpage as `.pdf`
- Or renamed `.txt` → `.pdf`

---

### 2. ❌ File is corrupted

- Interrupted download
- Copied incorrectly

---

### 3. ❌ It's not actually a PDF

Even if it says `.pdf`, it might be:

- `.txt`
- `.html`
- `.docx`

---

## ✅ Quick fix (do this now)

### Step 1: Open the file manually

- Double-click `sample.pdf`

👉 If it doesn't open in a PDF viewer → problem confirmed

---

### Step 2: Get a real PDF

Download something clean:

- Any textbook PDF
- Notes PDF

- Even a small 2–3 page file

---

### Step 3: Replace file

Keep same name:

```bash
```

Run again:

```bash
```

---

## 🛠️ Optional: Add error handling (good practice)

```
from pypdf import PdfReader
from pypdf.errors import PdfReadError

def extract_text_from_pdf(file_path):
    try:
        reader = PdfReader(file_path)
    except Exception as e:
        print(f"Error reading PDF: {e}")
        return ""

    text = ""

    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            text += page_text + "\n"

    return text
```

---

## 🧠 Important lesson (this matters later)

In real-world RAG systems:

- **Data quality > model quality**

Bad input → bad output
Always check your data first.

---

## 🎯 Your task

Fix the PDF and run again.

Then reply:

- ✅ "working"
- or send the new error

---

Once it works:

👉 we move to **chunking (Day 2)** — this is where things get interesting.

```
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf> python main.py
your pdf :
invalid pdf header: b'Simpl'
EOF marker not found
Traceback (most recent call last):
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf\main.py", line 21, in <module>
    main()
    ~~~~^^
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf\main.py", line 16, in main
    text=extract_text_from_pdf(file_path=filepath)
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf\main.py", line 5, in extract_text_from_pdf
    reader = PdfReader(file_path)
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 149, in __init__
    self._initialize_stream(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
```

```
line 171, in _initialize_stream
    self.read(stream)
    ~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 661, in read
    self._find_eof_marker(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 767, in _find_eof_marker
    line = read_previous_line(stream)
  File "C:\Python313\Lib\site-packages\pypdf\_utils.py", line 330, in read_previous_line
    raise PdfStreamError(STREAM_TRUNCATED_PREMATURELY)
pypdf.errors.PdfStreamError: Stream has ended unexpectedly
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf>
```

Yeah—this confirms it 100%:

👉 **Your file is NOT a real PDF (or it's corrupted).**
No amount of code changes will fix this.

---

## 🔍 Proof (based on your error)

```
invalid pdf header: b'Simpl'
```

A valid PDF **must start with**:

```
%PDF-1.x
```

Your file starts with:

```
Simpl...
```

That means:

- It's probably a **text/HTML file renamed to .pdf**
- Or a **bad download**

---

## 🧠 Quick verification (do this)

Open PowerShell in that folder and run:

```bash
```

👉 If you see readable text like:

```
Simple notes...
```

then it's **NOT a PDF**

---

## ✅ Fix (don't overthink)

### Option 1 (fastest)

Download a real PDF:

- Search: *"machine learning notes pdf"*
- Or any textbook PDF

---

### Option 2 (clean test file)

1. Open a browser
2. Press **Ctrl + P**
3. Save as PDF

Now you have a **valid PDF**

---

### Option 3 (rename trap check)

Make sure file is not:

```
sample.pdf.txt
```

Turn on extensions in Windows:

- View → ✔ File name extensions

---

## ⚠️ Important mindset

Right now you're debugging:

❌ code

But the real issue is:

✅ data

This happens *all the time* in real projects.

---

## 🧪 After fixing

Run again:

Bash

Expected:

- No crash ✅
- Text output ✅

---

## 🚫 Don't move forward yet

Chunking will fail if this isn't fixed.

---

## 🎯 Your move

Fix the file and reply:

- ✅ "**working**" → we go to chunking
- ❌ still error → paste again (we'll debug fast)

```
from pypdf import PdfReader
from pypdf.errors import PdfReadError

def extract_text_from_pdf(file_path):
    reader = PdfReader(file_path)
```

```
    text=""
    for page_num,page in enumerate(reader.pages):
        page_text=page.extract_text()
        if page_text:
            text+=page_text +'\n'
    return text

def main():
    filepath="sample.pdf"
    print("your pdf :")
    text=extract_text_from_pdf(file_path=filepath)
    print("your text is: \n ")
    print(text[:1000])

if __name__ == "__main__":
    main()
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf> python main.py
your pdf :
invalid pdf header: b'What '
EOF marker not found
Traceback (most recent call last):
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 21, in <module>
    main()
    ~~~~^^
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 16, in main
    text=extract_text_from_pdf(file_path=filepath)
  File "C:\Users\304508\Desktop\genai-daily-notes\day-02-
extract_text_from_pdf\main.py", line 5, in
extract_text_from_pdf
    reader = PdfReader(file_path)
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 149, in __init__
    self._initialize_stream(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 171, in _initialize_stream
    self.read(stream)
    ~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py",
line 661, in read
```

```
    self._find_eof_marker(stream)
    ~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Python313\Lib\site-packages\pypdf\_reader.py", line 767, in _find_eof_marker
    line = read_previous_line(stream)
  File "C:\Python313\Lib\site-packages\pypdf\_utils.py", line 330, in read_previous_line
    raise PdfStreamError(STREAM_TRUNCATED_PREMATURELY)
pypdf.errors.PdfStreamError: Stream has ended unexpectedly
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf>
```

Same error, different word:

```
invalid pdf header: b'What '
```

That's basically the file shouting:
👉 "I am NOT a PDF."

---

## 🧠 What's actually happening

Your file likely starts with something like:

```
What is machine learning...
```

Instead of:

```
%PDF-1.4
```

So again:

- ❌ Not a PDF
- ❌ Or badly saved file
- ❌ Or renamed `.txt/.html` → `.pdf`

---

## 🚫 Stop trying to fix code

Your code is fine.

This is 100% a **file issue**.

---

## 🔍 Do this (takes 10 seconds)

### Step 1: Check file type properly

Run:

```bash
```

If you see readable English text → **not a PDF**

---

### Step 2: Fix it properly (don't shortcut this)

✅ **Best option (clean test)**

1. Open any webpage
2. Press **Ctrl + P**
3. Select **Save as PDF**
4. Save as:

```bash
```

---

✅ **Alternative**

Download a real PDF from:

- college notes
- documentation
- ebooks

---

### Step 3: Replace your file

Keep same name:

```
sample.pdf
```

## ⚠️ Hidden mistake you might be doing

You might be:

> opening a `.txt` file → renaming it to `.pdf`

That **does NOT convert it**.

---

## 🧪 Final check (important)

Before running Python:

👉 Double-click the file

- Opens in browser/PDF viewer → ✅ good
- Opens in Notepad → ❌ bad

---

## 🎯 Expected result after fix

Run:

```bash
```

You should see:

- ✅ No crash
- ✅ Extracted text

---

## 🧠 Real-world lesson (don't ignore this)

You just learned:

> Debugging = checking assumptions

Your assumption:

> "I have a PDF"

Reality:

> You didn't

This is exactly how real bugs happen.

---

## ⏭️ Next

Fix the file and come back with:
👉 **"working"**

Then we go to:

## 🔥 Chunking (actual core of RAG)

PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf> type sample.pdf
What I recommend for you

Given your current level (you just built chatbot):

Do NOT pause for a separate project

Instead:

Build RAG like this:

Day 1 â†' PDF extraction (you did)
Day 2 â†' Chunking
Day 3 â†' Embeddings + FAISS (this = semantic search)
Day 4 â†' Retrieval
Day 5 â†' LLM response

Important mind
PS C:\Users\304508\Desktop\genai-daily-notes\day-02-extract_text_from_pdf>

There it is—caught red-handed 😄

👉 This is **not a PDF at all**.
It's just a **plain text file** with a `.pdf` extension.

That output:

```
What I recommend for you...
```

means:

- Windows thinks it's a PDF ❌
- But internally it's just a `.txt` file ✅

---

## 🧠 Why your code crashes

`pypdf` expects:

```
%PDF-1.x
```

But your file starts with:

```
What I recommend...
```

So it breaks with:

```
invalid pdf header
```

---

## ✅ Fix it properly (do this exactly)

### 🔥 Option 1 (fastest, do this)

1. Open **any webpage** (even this chat)
2. Press **Ctrl + P**
3. Select **Save as PDF**
4. Save as:

```Bash
```

---

### 🔥 Option 2 (also good)

Download a real PDF:

- Lecture notes

- Any ebook
- Even a 2-page doc

---

## ⚠️ Important rule

👉 **Renaming a file does NOT convert it**

```bash
```

You must actually **generate/download a real PDF**

---

## 🧪 Quick test before coding

Double-click `sample.pdf` :

- Opens in browser / PDF viewer → ✅ good
- Opens in Notepad → ❌ wrong file

---

## 🎯 After fixing

Run:

```bash
python main.py
```

You should finally see:

- ✅ No errors
- ✅ Extracted text

---

## 🧠 What you just learned (this matters a LOT)

Most beginners:

> blame code

Good engineers:

> verify input first

You just leveled up here.

---

## ⏭️ Next step

Once it works:

👉 say “**working**”

Then we move to:

## 🚀 Chunking (this is where RAG actually begins)